

TECHLOOM.AI

Software Engineer Intern - Practical Assessment

Duration	Submission	Tech Stack
72 Hours	Public GitHub Repo + Live Deployment Link	Any of: JavaScript, Node.js, React, Java, or Python

Important Note to Candidate

This assessment is divided into two sections. You are required to complete BOTH within 72 hours of receiving this document.

You may use any language/framework you are most comfortable with — JavaScript, Node.js, React, Java, or Python are all acceptable, for both backend and frontend where applicable. Pick the stack that lets you deliver your best, most complete work.

Each project must be pushed to a public GitHub repository AND deployed to a live, publicly accessible URL (e.g. Vercel, Netlify, Render, Railway, or similar). Submissions without a working deployment link will not be reviewed.

Assessment Overview

#	Section	Task
01	POS Order & Inventory System	Build a concurrency-safe POS backend with stock reservation, mock payments, and full order lifecycle handling
02	E-Commerce Checkout & Payment System	Build a mini storefront with search, checkout, stock reservation, mock payments, refunds, and order history

Background

Point-of-sale environments process many simultaneous transactions against the same product. This task evaluates your ability to design a system that manages product inventory, handles order creation under concurrent load, and reliably prevents overselling while gracefully handling payment outcomes.

Requirements

Product & Inventory Management

- Create, read, update, and delete products, each with a name, price, and available stock count
- Expose an endpoint or view that reflects the current, accurate stock level for every product

Cart & Order Creation

- Allow a user to add products to a cart and convert the cart into an order
- Validate stock availability before an order is allowed to proceed
- Prevent overselling when multiple users attempt to purchase the same limited-stock item at the same time

Stock Reservation

- Reserve stock for a product the moment a user enters checkout
- Expire the reservation automatically after 5 minutes if checkout is not completed
- Release reserved stock back to available inventory immediately on expiry

Mock Payment System

- Simulate a payment gateway with success, failure, and timeout outcomes
- Handle each outcome distinctly: confirm the order on success, release stock on failure, and expire the reservation on timeout
- Detect and reject duplicate payment or duplicate order submissions for the same cart or order

Order Lifecycle

- Support order cancellation, restoring stock correctly for any failed or cancelled order
- Model clear order statuses (e.g. Pending, Reserved, Paid, Cancelled, Expired, Failed) and enforce valid transitions between them
- Use database transactions where appropriate and handle errors without leaving inventory or orders in an inconsistent state

Technical Specifications

Backend	Any of: Node.js (Express/NestJS), Java (Spring Boot), or Python (FastAPI/Django) — candidate's choice
Frontend	React.js (or plain JavaScript if not using a framework)
Database	MongoDB or MySQL / PostgreSQL
Deployment	Must be deployed to a live, publicly accessible URL

Evaluation Criteria

Criterion	What We Look For
Concurrency Handling	No overselling occurs under simultaneous purchase requests
Reservation & Timeout	5-minute stock lock is correctly implemented and released on expiry
Payment Handling	Success, failure, timeout, and duplicate-submission cases are all handled correctly
Order Lifecycle	Statuses and transitions are consistent; cancellations correctly restore stock
Data Integrity	Database transactions prevent partial or inconsistent updates
Code Quality	Clean, modular, and well-documented code

Background

This task simulates a customer-facing online store. You will build the shopping experience end to end — from browsing and searching products through to checkout, mock payment, and post-purchase order management — with attention to how real payment gateways can fail or stall.

Requirements

Product Discovery

- Build a product listing page with search and filtering (e.g. by category, price range, or availability)
- Provide a product details view showing full information for a single product

Cart & Checkout

- Implement an add-to-cart and cart management flow
- Build a checkout flow that reserves stock for the items in the cart before payment is attempted

Mock Payment Gateway

- Simulate a payment gateway supporting successful payments, failed payments, and payment timeouts
- Prevent duplicate payment attempts from creating multiple charges or multiple orders for the same checkout session

Post-Purchase Flow

- Allow order cancellation and simulate a refund for cancelled or failed paid orders
- Provide an order history view for a user to see past orders and their statuses

Technical Specifications

Backend	Any of: Node.js, Java (Spring Boot), or Python — candidate's choice
Frontend	React.js or Next.js
Database	MongoDB or MySQL / PostgreSQL
Deployment	Must be deployed to a live, publicly accessible URL

Evaluation Criteria

Criterion	What We Look For
Discovery UX	Search and filtering return accurate, relevant results
Reservation Logic	Stock is reserved at checkout and released correctly if payment does not complete
Payment Handling	Success, failure, timeout, and duplicate-payment cases are all handled correctly
Refunds & Cancellation	Refund simulation and cancellation correctly reverse order and stock state

Criterion	What We Look For
Order History	Accurately reflects each order's current and past status
Code Quality	Clean, modular, and well-documented code

Submission Guidelines

Follow all steps before the 72-hour deadline

1	Push all code (Sections 01 and 02) to a single Public GitHub Repository
2	Deploy each project to a live hosting platform (e.g. Vercel, Netlify, Render, Railway, Fly.io) and record the working URL
3	Include a README.md with the tech stack, setup steps, environment variables, and how to test each feature
4	Add both deployment links plus the repository link clearly at the top of the README
5	Organise the repository into folders: /task-01, /task-02
6	(Optional) Record a brief screen walkthrough demo and include the link in your README
7	Share the repository link and live deployment links with the Techloom.ai team before your 72-hour deadline

✓ README.md Must include: tech stack, setup steps, live deployment links, and how to test each feature	✓ Repository Structure Organise into folders: /task-01, /task-02
--	--

A Note on Deployment

A GitHub repository alone is not sufficient — both tasks must be deployed and reachable at a live URL. Free-tier hosting (Vercel, Netlify, Render, Railway, Fly.io, etc.) is fine.

We are looking for candidates who can think independently, write clean code, and communicate their reasoning clearly. Quality matters more than quantity — a well-documented, focused, fully deployed submission will always outperform a rushed, incomplete one.

Good luck. The Techloom.ai Team